HRI Pipeline for Humanoid Robots in Household Scenarios

Software Documentation

version 1 (Dec 2025)

Abstract

This document provides a description of a Human–Robot Interaction (HRI) software pipeline contained in the accompanying repository. The pipeline integrates multi-modal sensing (vision, audio), perception (detection and pose estimation), and higher-level interaction logic. The system is spread across multiple ROS distributions and ROS 2, due to constraints imposed by hardware, contributions from individual authors and external code (sourced from git submodules). Because the system must interoperate with vendor and third-party components, it is deployed as containerized services with explicit bridges (ROS and ZeroMQ) between subsystems.

The documentation is split into a user-facing manual (installation, configuration, and operation) and a developer-facing software description (architecture, interfaces, extension points, and recommended practices).

Contents

1 Introduction **5**
1.1 Purpose and scope . . . 5
1.2 Repository structure (high-level) . . . 5
1.3 System diagrams and how to view them . . . 5

2 User Manual **5**
2.1 Supported platforms and prerequisites . . . 5
2.2 Installation . . . 5
2.2.1 System dependencies . . . 5
2.2.2 Repository checkout . . . 6
2.3 Configuration . . . 6
2.3.1 Environment variables . . . 6
2.3.2 Compose configuration . . . 6
2.4 Building and running . . . 6
2.4.1 Build images . . . 6
2.4.2 Start the pipeline . . . 6
2.4.3 Selective execution with profiles . . . 7
2.5 Operational workflow . . . 7
2.6 Troubleshooting . . . 7
2.6.1 GPU not visible inside container . . . 7
2.6.2 ROS networking issues . . . 7

3 Software Documentation **7**
3.1 Architecture overview . . . 7
3.2 Communication and integration . . . 8
3.2.1 ROS topics . . . 9
3.2.2 ROS services . . . 10
3.2.3 ZMQ bridges . . . 10
3.3 Code structure and key classes . . . 10
3.3.1 Interfaces (members) . . . 11
3.4 Communication and Data Flow . . . 11
3.4.1 Inputs and sensor interfaces . . . 12
3.4.2 Perception pipeline: detection → pose estimation . . . 12
3.4.3 ZMQ bridge: packaging perception into cross-version payloads . . . 12
3.4.4 Gesture and microphone processing . . . 13
3.4.5 Reasoning layer: intent fusion and action selection . . . 13
3.4.6 Execution layer: routing actions to robot control . . . 13
3.4.7 End-to-end summary . . . 13
3.5 Extension points and best practices . . . 14
3.5.1 Adding a new perception component . . . 14
3.5.2 Improving robustness . . . 14
3.6 Testing and validation . . . 14

List of Figures

1 Conceptual system overview. 8
2 Communication diagram (ROS topics/services and ZMQ endpoints). 9
3 Class diagram and interface overview for node and ZMQ classes. 11

List of Tables

1 Introduction

1.1 Purpose and scope

The goal of the HRI pipeline is to provide a modular, containerized software stack for humanoid-robot human–robot interaction tasks. The software is designed to integrate several subsystems:

- perception from cameras and microphones,
- real-time object detection and 6D pose estimation,
- gesture / pointing recognition,
- speech and audio pre-processing,
- communication with vendor robot stacks,
- cross-version bridging between ROS 1 and ROS 2 components.

Because different subsystems were delivered in different ROS distributions (e.g., ROS 1 Noetic, ROS 1 Melodic, ROS 2), and because vendor stacks may impose additional constraints, the repository uses explicit bridging and containerization to standardize deployment and isolate dependencies.

1.2 Repository structure (high-level)

The repository follows the pattern:

- `compose/` — Docker / Docker Compose configuration and image definitions,
- `src/` — ROS workspaces and ROS packages (Noetic/Melodic/ROS 2),
- `lib/` — shared helper libraries (including ZMQ utilities),
- `config/` — YAML/JSON configuration used by nodes,
- `data/` — datasets, weights, and runtime assets.

1.3 System diagrams and how to view them

This documentation references three diagrams:

1. **Conceptual overview** (high-level functional blocks),
2. **Communication diagram** (ROS topics, services, ZMQ endpoints),
3. **Class + communication diagram** (node classes and interfaces).

2 User Manual

2.1 Supported platforms and prerequisites

The software is intended to run on Linux (Ubuntu recommended), with NVIDIA GPU acceleration for perception components. Prerequisites:

- Docker Engine and Docker Compose (Compose V2 plugin recommended),
- NVIDIA drivers and NVIDIA Container Toolkit for GPU containers,
- network access to the robot or simulation stack,

2.2 Installation

2.2.1 System dependencies

Install Docker and the NVIDIA Container Toolkit using the official documentation for your distribution. After installation:

Listing 1: Verify Docker and GPU access

```
docker --version
docker compose version
nvidia-smi
```

```
docker run --rm --gpus all nvidia/cuda:12.2.0-base-ubuntu22.04
    nvidia-smi
```

2.2.2 Repository checkout

Listing 2: Clone repository and initialize submodules

```
git clone git@github.com:ichores-research/hri-pipeline.git
cd hri-pipeline
git submodule update --init --recursive
```

2.3 Configuration

2.3.1 Environment variables

The pipeline expects environment variables for ROS master location, robot IP, dataset selection, and configuration selection. Recommended practice is to keep these in a `.env` file.

Listing 3: Example .env file

```
# ROS master
ROS_MASTER_URI=http://127.0.0.1:11311
ROS_IP=127.0.0.1

# Robot-side master (if bridging to vendor stack)
ROBOT_ROS_MASTER_URI=http://<ROBOT_HOSTNAME_OR_IP>:11311

# Dataset / weights selection
DATASET=ycbv
CONFIG=default.yaml
```

2.3.2 Compose configuration

Docker Compose files define services such as detection, pose estimation, gesture recognition, audio processing, and ROS bridge nodes. GPU-enabled services specify `runtime: nvidia` or the newer `deploy.resources.reservations.devices` syntax.

Tip: keep host-specific values out of the compose file; use `.env` and YAML anchors to reuse common environment blocks.

2.4 Building and running

2.4.1 Build images

Listing 4: Build all images

```
docker compose build
```

2.4.2 Start the pipeline

Listing 5: Start all services

```
docker compose up
```

For interactive debugging or to observe logs per node, recommended approaches include:

- `docker compose logs -f <service>`
- a tmux-based launcher that attaches to each service shell / exec,
- splitting the compose file into profiles (`profiles:`) for selective startup.

2.4.3 Selective execution with profiles

If services are grouped into profiles, you can run only a subset:

Listing 6: Run only perception services

```
docker compose --profile perception up
```

2.5 Operational workflow

A typical runtime sequence is:

1. bring up ROS master / vendor stack,
2. start bridges (ROS1–ROS2 and/or ZMQ),
3. start perception services (detectors / pose estimators),
4. start HRI logic (gesture/speech/intent),
5. validate topics and services (`rostopic list`, `rosservice list`).

2.6 Troubleshooting

2.6.1 GPU not visible inside container

Check that:

- NVIDIA drivers are installed (`nvidia-smi`),
- NVIDIA Container Toolkit is installed,
- Compose service uses `-gpus` / `runtime: nvidia` / device reservation.

2.6.2 ROS networking issues

Common causes:

- incorrect `ROS_MASTER_URI` or `ROS_IP`,
- conflicting host networking, firewalls,
- multiple ROS masters.

Use `network_mode: host` only when required. When bridging, ensure each container uses consistent networking assumptions.

3 Software Documentation

3.1 Architecture overview

The pipeline architecture can be viewed at three levels:

1. **Conceptual:** high-level functional blocks and data flow.
2. **Communication:** concrete ROS topics/services and ZMQ endpoints.
3. **Code-level:** node classes, helper classes, and interface methods.

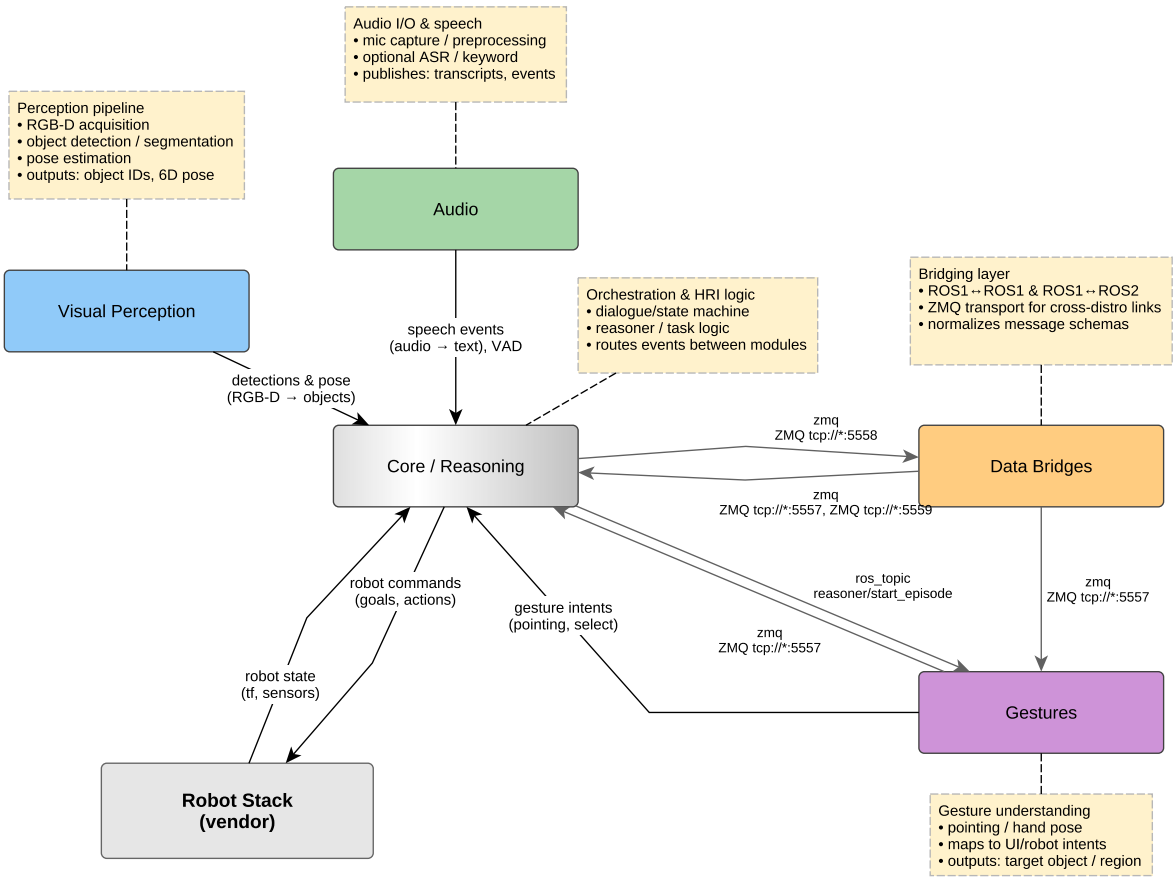

Figure 1: Conceptual system overview.

3.2 Communication and integration

The system uses ROS topics and services inside each ROS distribution, and explicit bridges between subsystems. In addition to ROS-native communication, the code base uses ZMQ-based channels to exchange structured data (often JSON dictionaries serialized as strings). These serve for communication between different ROS distros (e.g., transmission of action server data or TFs).

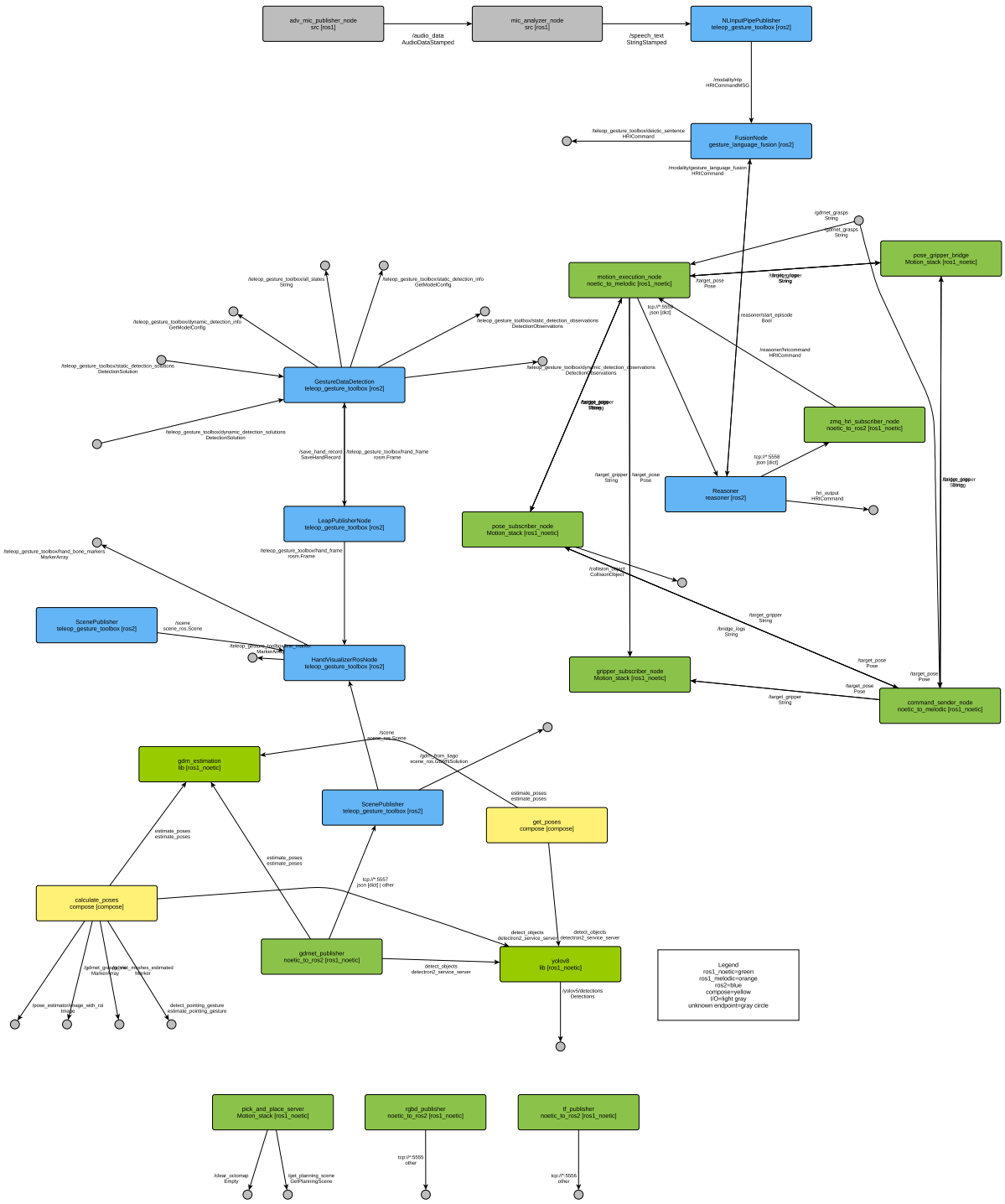

Figure 2: Communication diagram (ROS topics/services and ZMQ endpoints).

3.2.1 ROS topics

ROS topics are used for streaming data such as:

- RGB and depth images,
- detection outputs (bounding boxes, masks),
- pose estimates (position/orientation in camera frame),
- gesture and pointing estimates,
- robot state, teleoperation commands.

3.2.2 ROS services

Services are used for request/response interactions such as:

- object detection on-demand,
- pose estimation for a given detection and image,
- configuration queries.

3.2.3 ZMQ bridges

ZMQ is used when:

- a ROS version mismatch prevents direct topic/service bridging,
- structured payloads are easier to serialize as dictionaries,
- performance requires decoupled IPC patterns.

Recommended convention: Prefer JSON payloads (`send_json`) with an explicit schema (keys, units, coordinate frames). If sending stringified dicts, document the key set and version it. Dictionary validation would be even better but this SW stack does not include such validation (possible future extension).

3.3 Code structure and key classes

Node implementations follow ROS conventions:

- ROS 1 nodes are typically scripts or classes instantiated in a script.
- ROS 2 nodes derive from `rclpy.node.Node`.
- bridge utilities encapsulate ZMQ send/recv patterns.

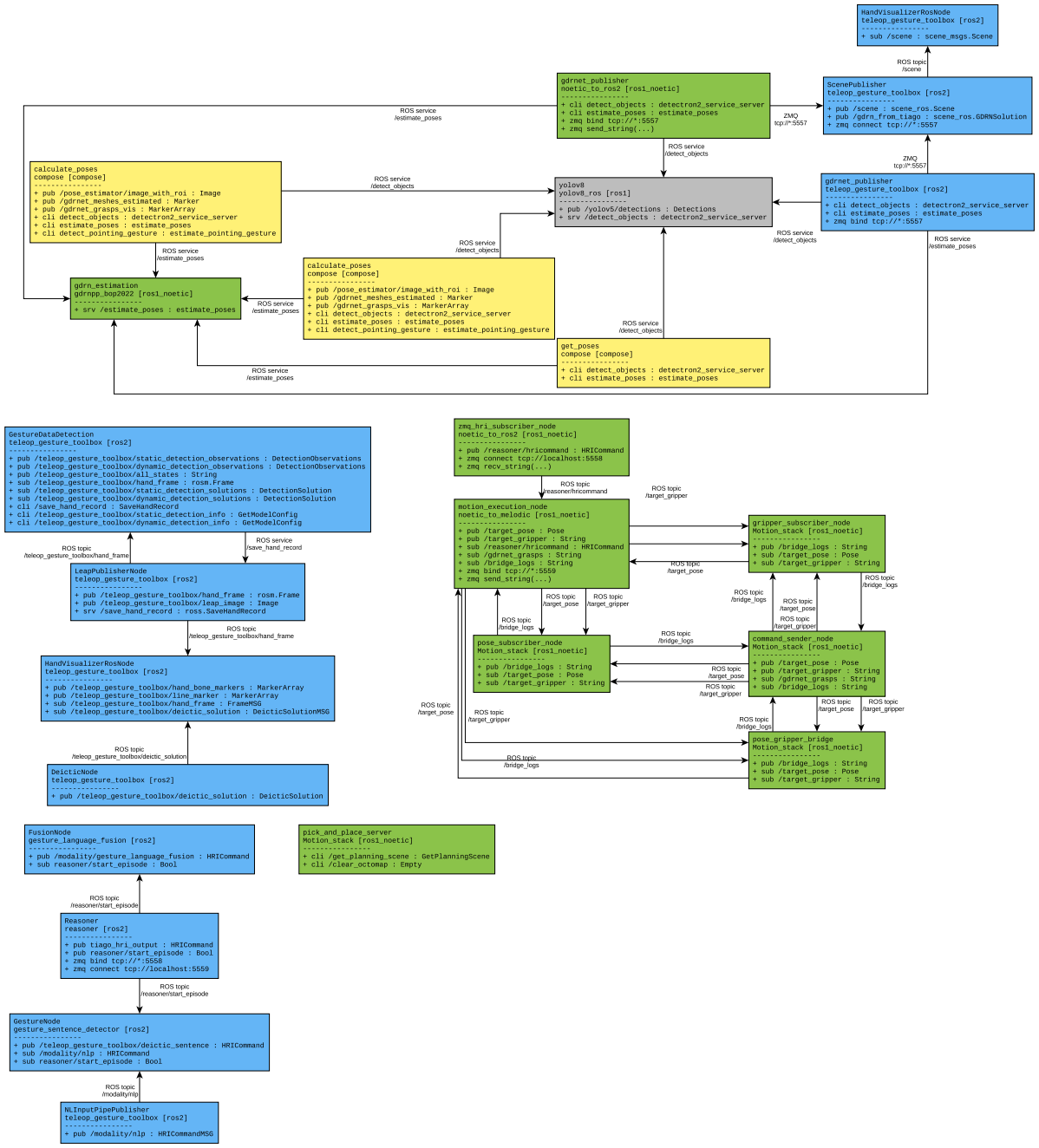

Figure 3: Class diagram and interface overview for node and ZMQ classes.

3.3.1 Interfaces (members)

Each node exposes a set of interfaces:

- **Publishers:** topics the node publishes to.
- **Subscribers:** topics the node consumes.
- **Service servers/clients:** request-response interfaces.
- **ZMQ endpoints:** bind/connect sockets and send/recv methods.

3.4 Communication and Data Flow

This section explains how data flows through the HRI pipeline. Figure 2 shows the complete communication graph; this section describes the end-to-end flow from sensor inputs through

perception and reasoning into robot execution.

3.4.1 Inputs and sensor interfaces

The system consumes sensory information primarily from:

- **Camera RGB and depth streams** (published as `sensor_msgs/Image` and `sensor_msgs/CameraInfo`).
- **Microphone / audio stream** (raw audio).
- **Robot state and transforms** from the vendor stack (TF tree, joint states, robot pose).

These inputs originate either from physical drivers on the robot and sensors or from a simulation, and are visible inside the relevant container via the configured `ROS_MASTER_URI`.

3.4.2 Perception pipeline: detection → pose estimation

The core perception pipeline is implemented as a service-based flow. A node responsible for orchestrating perception calls the following ROS services:

`detect_objects` service (2D detection). A client node creates a service proxy:

```
detect_objects = rospy.ServiceProxy('detect_objects',
    detectron2_service_server)
response = detect_objects(rgb)
```

The service returns a set of 2D detections (bounding boxes / segmentation / scores). These detections are used as input for the pose estimation step.

`estimate_poses` service (6D pose estimation). The same or downstream node then requests pose estimation:

```
estimate_pose = rospy.ServiceProxy('estimate_poses', estimate_poses)
response = estimate_pose(detection, rgb, depth)
```

The returned output contains 6D pose estimates (position + orientation) and include additional metadata (confidence, object ID, covariance). These are the primary world-state measurements used by the HRI logic.

Pose publication into TF tree (transform_pub). The output poses are published into the ROS TF tree via a transform publisher node (`transform_pub`). This makes poses globally usable by any node (navigation, manipulation, gesture reasoning). The TF interface is therefore the *canonical* representation of object pose in the system.

3.4.3 ZMQ bridge: packaging perception into cross-version payloads

Several nodes serialize the perception results and transmit them over ZeroMQ to allow cross-ROS-version transport and decouple the reasoner from the perception stack.

`zmq_gdrnet_publisher` (object dictionary sender). In the current system, `zmq_gdrnet_publisher` sends object data as a dictionary serialized into a string payload. The payload includes:

- object ID / class label,
- 6D pose estimation output (translation, rotation),
- timestamps and frame IDs.

This node receives pose information indirectly via the services described above (`detect_objects`, `estimate_poses`). The ZMQ message therefore represents a condensed and transport-neutral "object state" package that can be consumed by the reasoner regardless of ROS distribution.

ZMQ receivers. On the receiving side, a ZMQ subscriber/receiver node deserialize the payload back into structured Python objects and republish the relevant parts as ROS topics or internal state to the reasoning layer.

3.4.4 Gesture and microphone processing

In parallel with object perception, the pipeline runs gesture perception:

`pointing_gesture_recognition` node. This node subscribes to the camera topics configured via ROS parameters (e.g., `/pose_estimator/color_topic`, `/pose_estimator/depth_topic`). Its output is a pointing/gesture hypothesis which include:

- gesture class (currently, only pointing is used),
- 3D pointing ray or target direction,
- confidence score,
- optional target object ID inferred from geometry.

This information is later fused with object pose information to determine which object the human refers to.

`mic_analyzer_node` and related audio processing. The microphone pipeline (e.g., `mic_analyzer_node`) processes audio input - detection voice presence in the stream (VAD) and if speech is detected, transcribes it into text. NL node then transforms this text into a structured NL command. The outputs are used by the reasoner as a second modality for intent inference.

3.4.5 Reasoning layer: intent fusion and action selection

The **Reasoner** is the central decision-making node that fuses information from:

- object pose hypotheses (from pose estimation via TF and ZMQ payloads),
- gesture/pointing hypotheses,
- speech hypothesis,
- robot state and task context.

The Reasoner maintains a world model (objects + poses) and estimates the most likely user intent (e.g., "pick that object", "move the banana here"). It then produces an execution-level command.

3.4.6 Execution layer: routing actions to robot control

The execution layer translates Reasoner decisions into robot actions. These actions are sent through:

- vendor ROS topics/services (navigation and manipulation),
- teleoperation toolboxes (e.g., `teleop_gesture_toolbox` configuration and dynamic model services),
- TF updates for targets and motion planners.

In deployments where the vendor stack runs on a separate ROS master or in a different ROS distribution, execution commands may be bridged through a ROS1–ROS1 bridge (e.g., Noetic $\leftrightarrow$ Melodic) or via ZMQ where appropriate.

This system was developed on the PAL Robotics Tiago robot running ROS 1 Melodic. The included Motion stack is tuned for this specific use-case. However, the motion stack can be adapted or replaced to fit any suitable robot.

3.4.7 End-to-end summary

In the current repository, the dominant end-to-end loop is:

Camera RGB-D → `detect_objects` (service) → `estimate_poses` (service) → `transform_pub` (TF) → `zmq_gdrnet_publisher` (dict payload) → **Reasoner** (fusion) → **Robot execution**

Meanwhile, the gesture and microphone branches feed the Reasoner in parallel:

Camera → `pointing_gesture_recognition` → Reasoner
Microphone → `mic_analyzer_node` → Reasoner

This design is intentionally modular and containerized to tolerate the multi-ROS-version constraint, but it requires strict interface discipline: consistent topic/service names, stable ZMQ payload schemas, and robust startup ordering.

3.5 Extension points and best practices

3.5.1 Adding a new perception component

To integrate a new detector or pose estimator:

1. implement as a standalone node/service in an isolated container,
2. define its inputs/outputs as ROS topics or services,
3. if cross-version bridging is needed, wrap the interface in a ZMQ bridge with versioned JSON schema,
4. add the service to Docker Compose and assign appropriate profiles.

3.5.2 Improving robustness

Due to the inherent complexity of multi-ROS systems:

- enforce explicit startup ordering (health checks, retries),
- use message timestamps and frame IDs consistently,
- centralize configuration (YAML + environment variables),
- validate payload schemas on ZMQ boundaries,
- add monitoring (ROS diagnostics, structured logs).

3.6 Testing and validation

Recommended validation steps:

- unit test utility libraries (serialization, math, transforms),
- integration test containers via `docker compose up` in CI,
- runtime smoke test: verify topic/service availability and basic messages,
- benchmark GPU services for latency and memory usage.

References

[1] ROS Wiki. https://wiki.ros.org/

[2] Docker Documentation. https://docs.docker.com/

[3] NVIDIA Container Toolkit. https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/

[4] ZeroMQ Guide. https://zguide.zeromq.org/